

Dr. Mahmoud Alnamoly

x

06

Queue

Dr. Mahmoud Alnamoly

x

+

Dr. Mahmoud Alnamoly

Queue ?

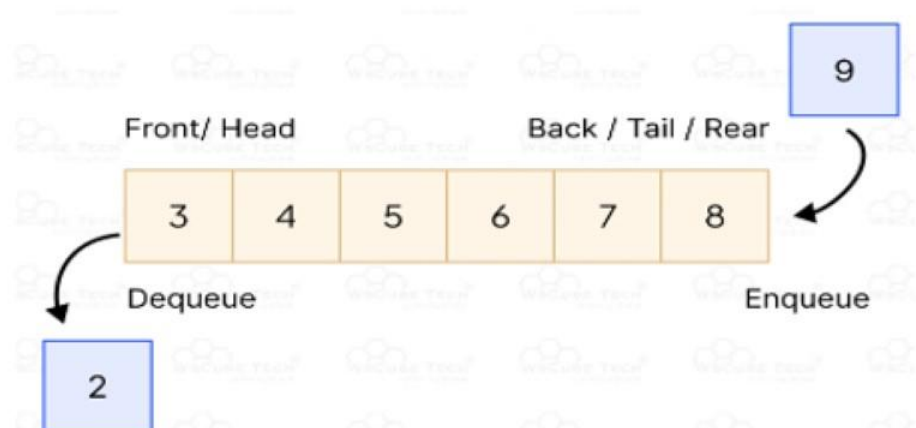
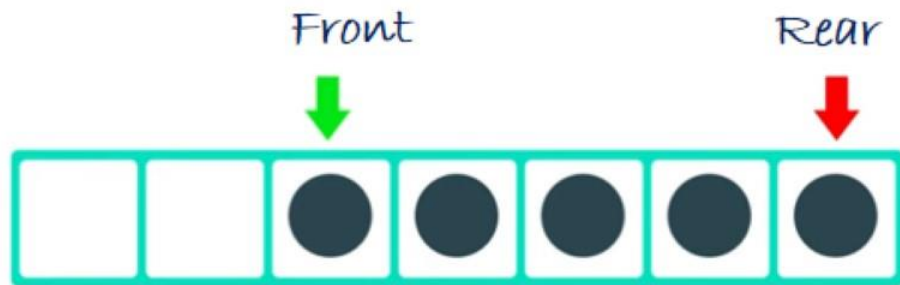
A queue in data structures is a linear collection of elements that operates under the **First In, First Out (FIFO)** principle. This means that the first element added to the queue will be the first one removed.



Types of Queue Data Structure

1. Simple Queue

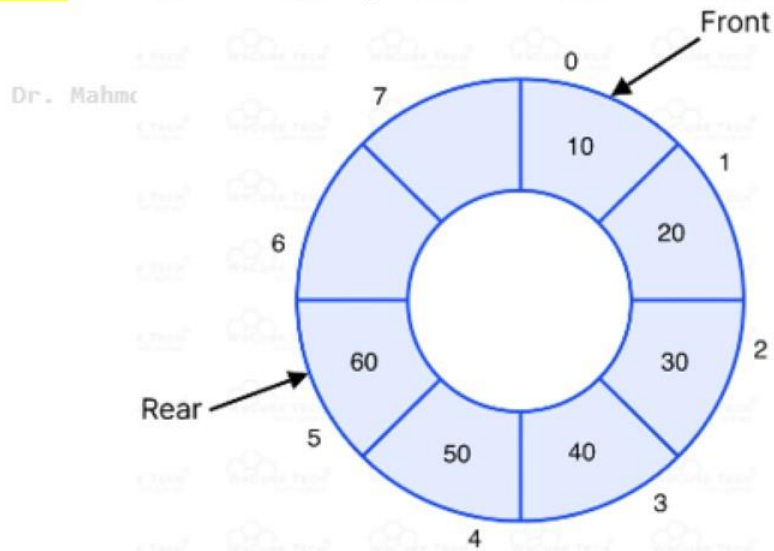
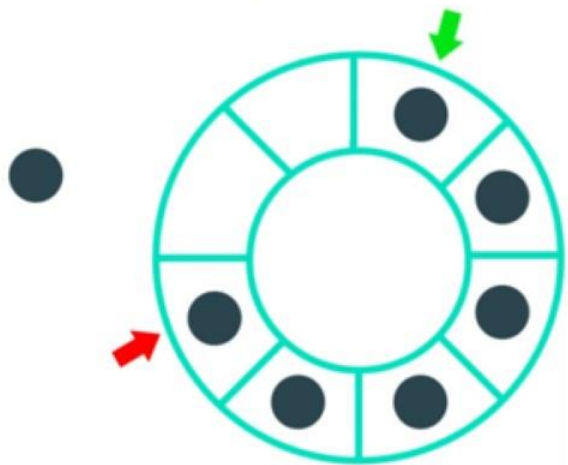
A simple queue operates on the FIFO (First In, First Out) principle, where elements are **added** to the **rear** and **removed** from the **front**. It is the most basic form of a queue used for sequential data processing.



2. Circular Queue

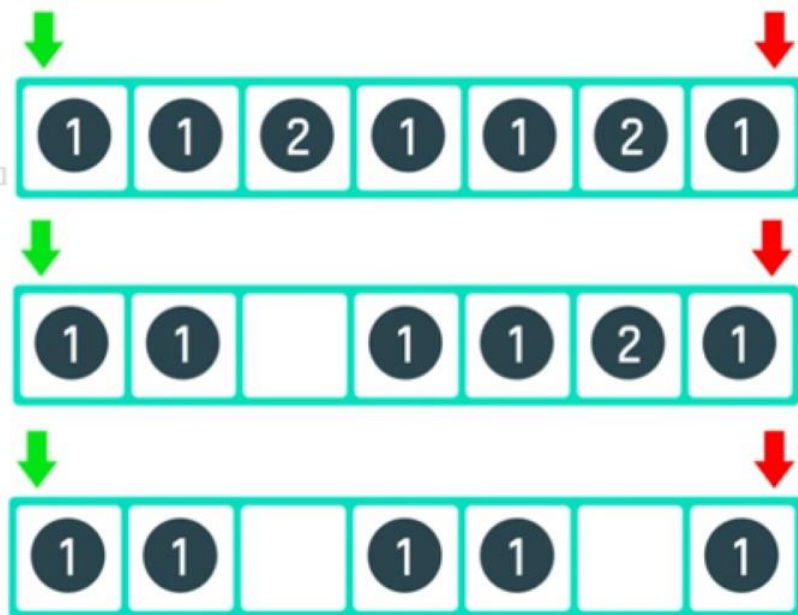
A circular queue is a more efficient version of the simple queue. It uses a circular structure to utilize memory optimally.

In a circular queue, when the rear reaches the end of the array and there is space at the beginning, it can **wrap around** and use this space, thus maximizing the use of available memory.



3. Priority Queue

- ✓ In a priority queue, elements are **not necessarily dequeued** in the **order** they were enqueued.
- ✓ Instead, each element is associated with a **priority**, and the element with the **highest priority** is dequeued first.

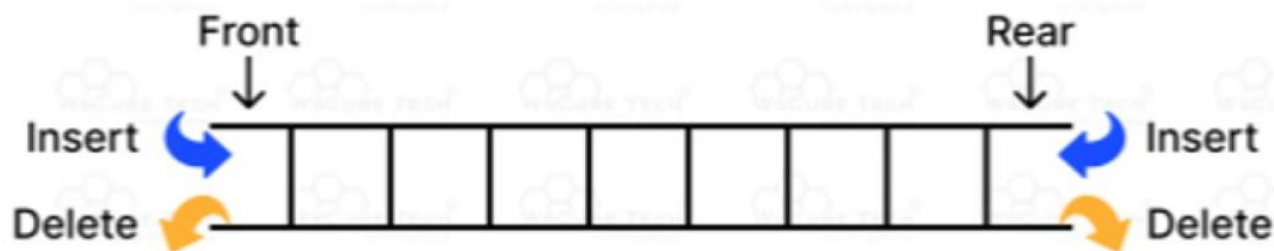


- ✓ Priority queues are often implemented using **heaps** to allow for efficient priority-based retrieval.

4. Double-Ended Queue (Deque)

- ✓ A deque allows insertion and removal of elements from both the front and the rear of the queue. This flexibility makes it a handy tool for various applications where elements need to be processed from both ends.
- ✓ Example: A list of recently used documents or applications where you can add new entries from both ends, or remove the least recently used items from either end, based on user interaction.

Dr. Mahmoud Alnamoly



Simple Queue

- ✓ In programming terms, putting items in the queue is called **enqueue**, and removing items from the queue is called **dequeue**.



Basic Operations of Queue

- ✓ **Enqueue**: Add an element to the end of the queue
- ✓ **Dequeue**: Remove an element from the front of the queue

Another operations

- ✓ **Peek(Getfront)**: Get the value of the front of the queue without removing it
- ✓ **IsEmpty**: Check if the queue is empty
- ✓ **IsFull**: Check if the queue is full
- ✓ **Getsize**: Return number of items in queue

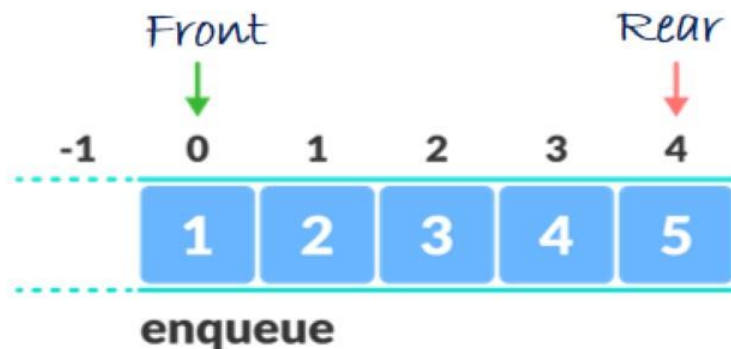
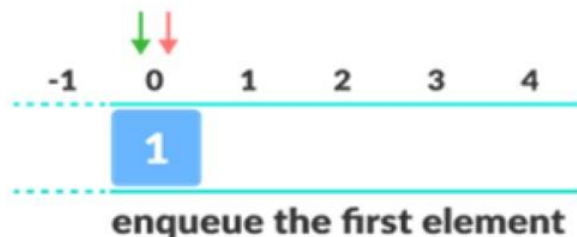
Basic Operations of Queue

✓ **Enqueue:** Add an element to the end of the queue



empty queue

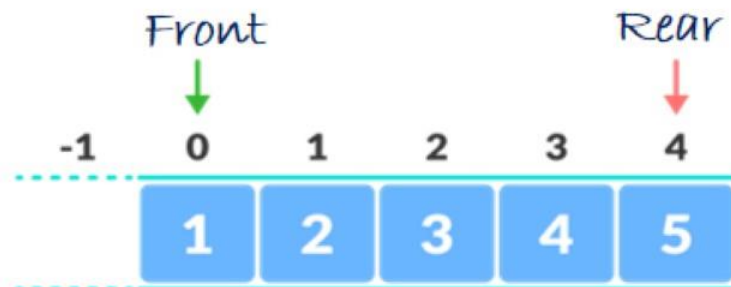
Dr. Mahmoud Alnamoly



$\text{rear} == \text{capacity} - 1$

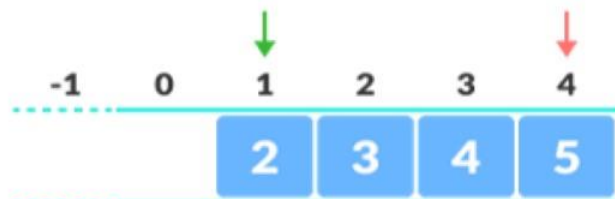
Basic Operations of Queue

✓ **Dequeue:** Remove an element from the front of the queue

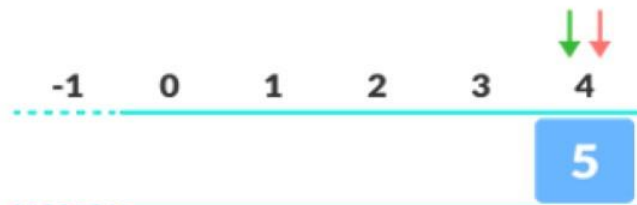


enqueue

Dr. Mahmoud Alnamoly



dequeue



dequeue the last element

Front Rear



empty queue

front == -1 || Front > rear

```
// Define a class for Queue
class Queue {
private:
    int front, rear, capacity;
    int* queue;

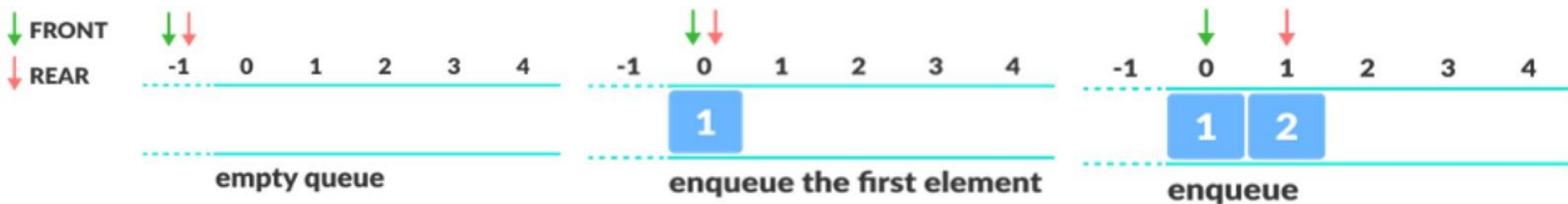
public:
    // Constructor to initialize the queue
    Queue(int cap) {
        capacity = cap;
        queue = new int[cap];
        front = -1;
        rear = -1;
    }
```

```
    // Destructor to free the memory allocated for the queue
    ~Queue() {
        delete[] queue;
    }
```

✓ **Enqueue: Add an element to the end of the queue**

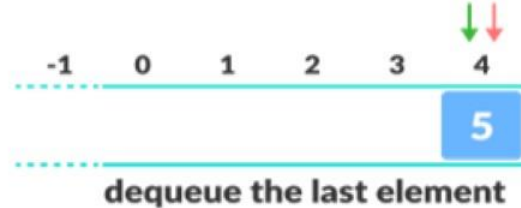
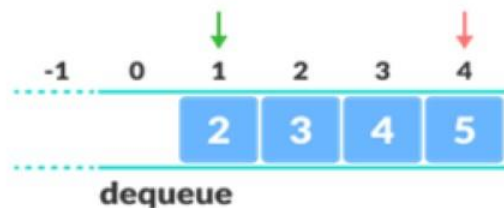
Dr. Mahmoud Alnamoly

```
void enqueue(int value) {  
    if (isFull()) {  
        cout << "Queue is full. Cannot enqueue " << value << endl;  
        return;  
    }  
    if (front == -1) front = 0; // First element enqueued  
    queue[++rear] = value;  
    cout << value << " enqueued to the queue.\n";  
}
```



✓ **Dequeue: Remove an element from the front of the queue**

```
void dequeue() {  
    if (isEmpty()) {  
        cout << "Queue is empty. Cannot dequeue.\n";  
        return;  
    }  
    cout << queue[front] << " dequeued from the queue.\n";  
    front++;  
    // Reset the queue if all elements are dequeued  
    if (front > rear) {  
        front = rear = -1;  
    }  
}
```

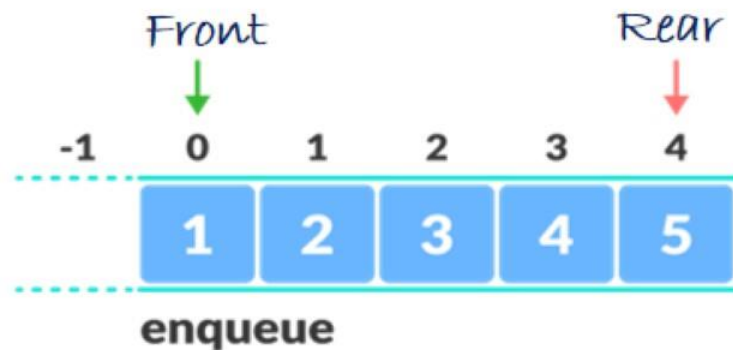


✓ **Peek: Get the value of the front of the queue without removing it**

Dr. Mahmoud Alnamoly

```
int getFront() {  
    if (isEmpty()) {  
        cout << "Queue is empty.\n";  
        return -1;  
    }  
    return queue[front];  
}
```

Dr. Mahmoud Alnamoly



✓ **IsEmpty: Check if the queue is empty**

```
bool isEmpty() {  
    return front == -1;  
}
```



dequeue the last element

Front Rear



✓ **IsFull: Check if the queue is full**

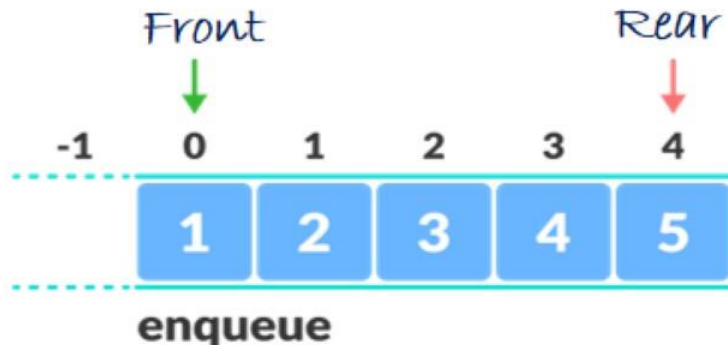
```
bool isFull() {  
    return rear == capacity - 1;  
}
```



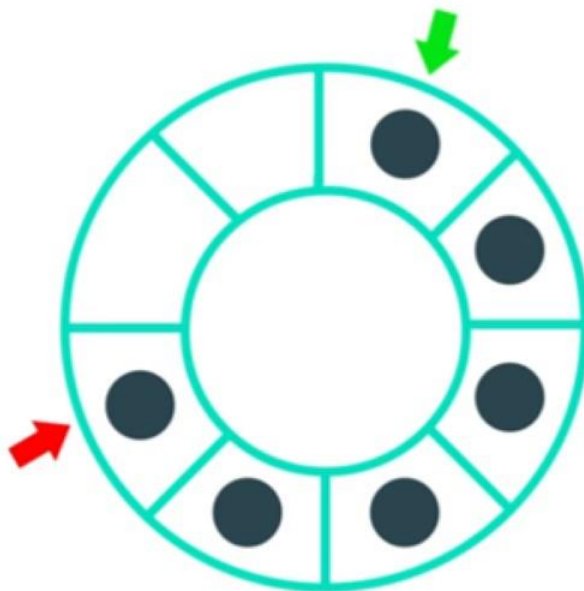
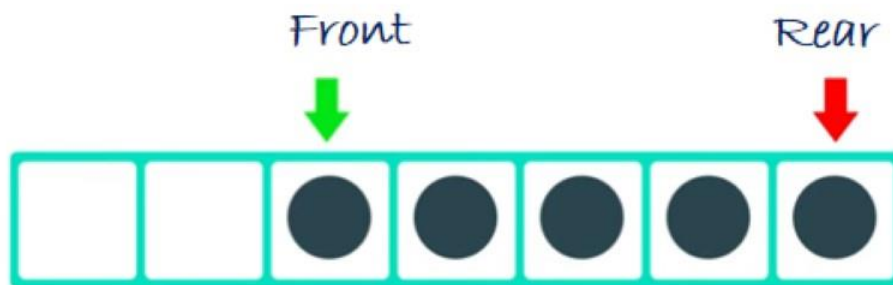
✓ **display: Print queue elements**

Dr. Mahmoud Alnamoly

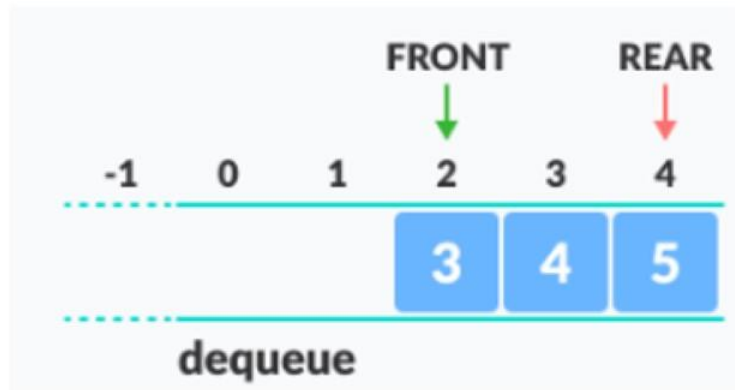
```
void display() {  
    if (isEmpty()) {  
        cout << "Queue is empty.\n";  
        return;  
    }  
    cout << "Queue elements: ";  
    for (int i = front; i <= rear; i++) {  
        cout << queue[i] << " ";  
    }  
    cout << endl;  
}
```



Circular Queue



- ✓ The circular queue solves the major limitation of the normal queue. In a normal queue, after a bit of insertion and deletion, there will be non-usable empty space.



- ✓ Circular Queue works by the process of **circular increment**
- ✓ i.e. when we try to increment the pointer (**rear/front**) and we reach the end of the queue, we start from the beginning of the queue.
- ✓ Here, the circular increment is performed by modulo division with the queue size. That is,

Dr. Mahmoud **`rear = (rear + 1) % capacity; // Circular increment`**

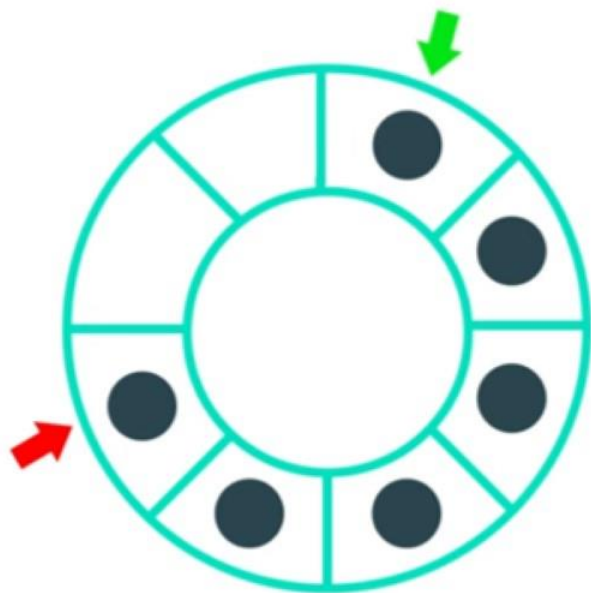
Front



Rear

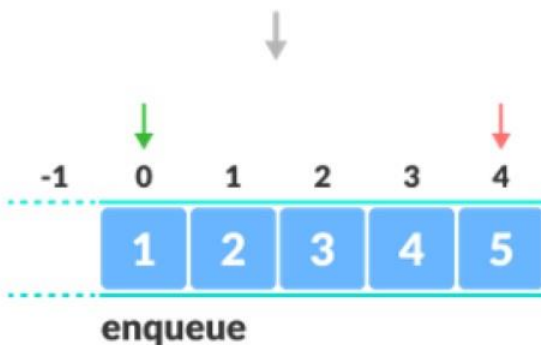
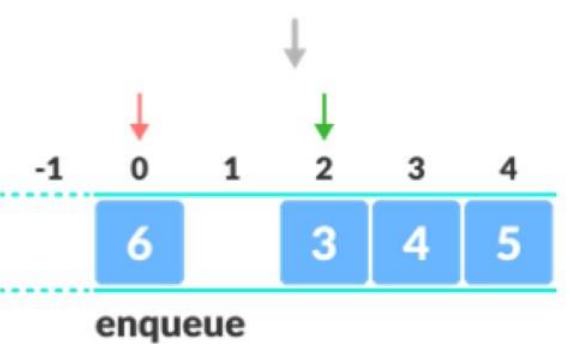
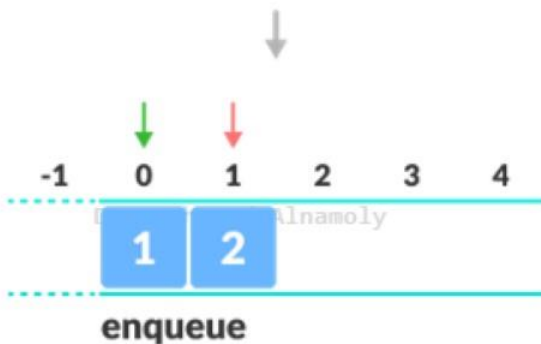
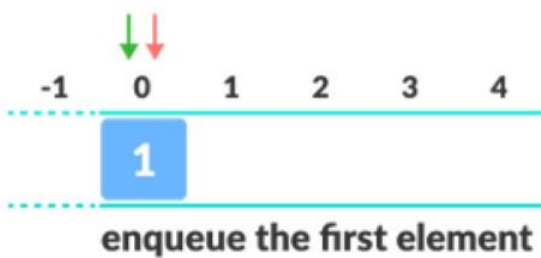
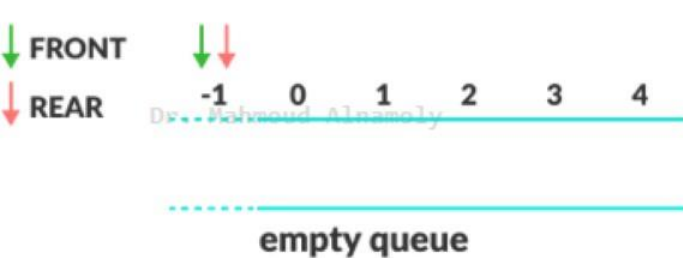


```
// Normal increment  
rear = (rear + 1);
```



Dr. Mahmoud Alnamoly

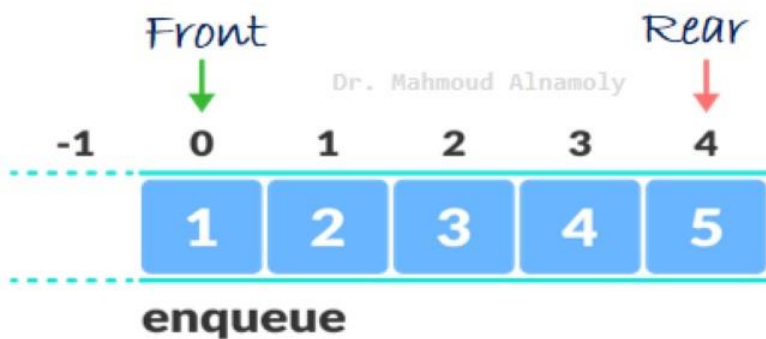
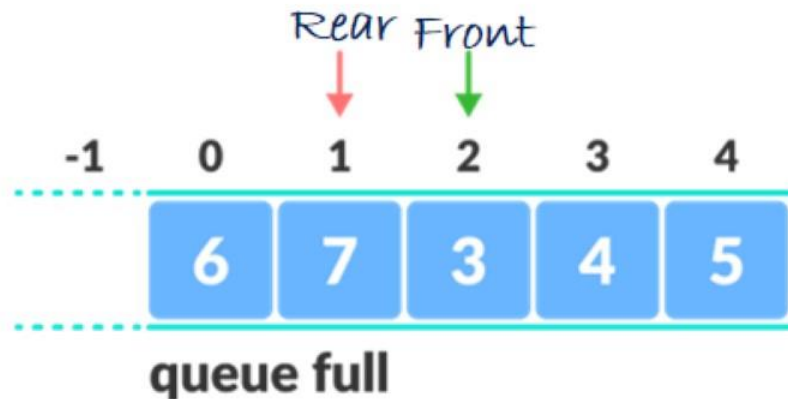
```
// Circular increment  
rear = (rear + 1) % capacity;  
rear = (rear + 2) % capacity;  
rear = (rear + 3) % capacity;  
rear = (rear + 4) % capacity;  
  
rear = (rear + i) % capacity;
```



Note:-

Front > rear

**Not indicator for
empty circular queue
like simple queue**



IsFull

$(rear + 1) \% capacity == front$

Implementation of Circular Queue

Dr. Mahmoud Alnamoly

```
#include <iostream>
#include <stdexcept> // For std::runtime_error

template <typename T>
class Queue {
private:
    T* arr;           // Array to hold the queue elements
    int capacity;     // Maximum capacity of the queue
    int front;        // Index of the front element
    int rear;         // Index of the rear element
    int size;         // Current size of the queue
```

Dr. Mahmoud Alnamoly

Implementation of Circular Queue

Dr. Mahmoud Alnamoly

```
public:
    // Constructor
    Queue(int cap = 10) : capacity(cap), front(0), rear(-1), size(0) {
        arr = new T[capacity]; // Allocate array
    }

    // Destructor
    ~Queue() {
        delete[] arr; // Free allocated memory
    }
```


Implementation of Circular Queue

Dr. Mahmoud Alnamoly

// Add an element to the rear of the queue

```
void enqueue(const T& value) {  
    if (size == capacity) {  
        resize(); // Resize if full  
    }  
    rear = (rear + 1) % capacity; // Circular increment  
    arr[rear] = value;           // Insert element  
    size++;  
}
```

Dr. Mahmoud Alnamoly

Implementation of Circular Queue

Dr. Mahmoud Alnamoly

// Remove and return the front element of the queue

```
T dequeue() {  
    if (isEmpty()) {  
        throw std::runtime_error("Queue is empty. Cannot dequeue.");  
    }  
    T value = arr[front];  
    front = (front + 1) % capacity; // Circular increment  
    size--;  
    return value; // Return the dequeued value  
}
```

// Peek at the front element without removing it

```
T peek() const {  
    if (isEmpty()) {  
        throw std::runtime_error("Queue is empty. Cannot peek.");  
    }  
    return arr[front]; // Return the front value  
}
```

Dr} Mahmoud Alnamoly

Implementation of Circular Queue

Dr. Mahmoud Alnamoly

```
// Check if the queue is empty
```

```
bool isEmpty() const {  
    return size == 0;  
}
```

```
// Get the number of elements in the queue
```

```
int getSize() const {  
    return size;  
}
```

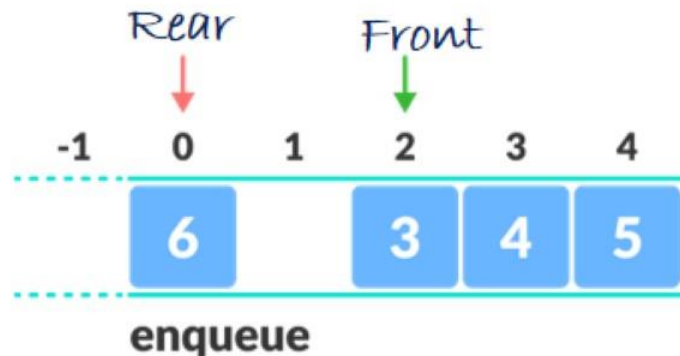
Dr. Mahmoud Alnamoly

```
// Clear the queue
```

```
void clear() {  
    front = 0;  
    rear = -1;  
    size = 0;  
}
```

Implementation of Circular Queue

Dr. Mahmoud Alnamoly



```
// Display elements in the queue
```

```
void display() const {  
    if (isEmpty()) {  
        std::cout << "Queue is empty." << std::endl;  
        return;  
    }  
    std::cout << "Queue elements: ";  
    for (int i = 0; i < size; ++i) {  
        std::cout << arr[(front + i) % capacity] << " ";  
    }  
    std::cout << std::endl;  
}
```

Dr. Mahmoud Alnamoly

Implementation of Circular Queue

```
// Resize the queue when full
void resize() {
    int newCapacity = capacity * 2; // Double the capacity
    T* newArr = new T[newCapacity];

    for (int i = 0; i < size; ++i) {
        newArr[i] = arr[(front + i) % capacity]; // Copy elements to new array
    }

    delete[] arr; // Free old array
    arr = newArr; // Point to new array
    front = 0;    // Reset front
    rear = size - 1; // Update rear
    capacity = newCapacity; // Update capacity

    std::cout << "Queue resized to capacity: " << capacity << std::endl;
}
```

Dr. Mahmoud Alnamoly

Output of code

Dr. Mahmoud Alnamoly

```
int main() {  
    //Create integers queue with capacity of 5  
    Queue<int> queue(5);  
    // Enqueue elements  
    queue.enqueue(10);  
    queue.enqueue(20);  
    queue.enqueue(30);  
    queue.enqueue(40);  
    queue.enqueue(50);  
    // Display queue elements  
    queue.display();  
  
    std::cout << "Front element: " <<  
queue.peek() << std::endl; // Output: 10  
    std::cout << "Queue size: " <<  
queue.getSize() << std::endl; // Output: 5 }  
}
```

```
// Dequeue elements
```

```
std::cout << "Dequeued: " <<  
queue.dequeue() << std::endl; // Output:  
10
```

```
queue.display(); // Display queue  
elements after dequeue
```

```
// Peek again
```

```
std::cout << "Front element after  
dequeue: " << queue.peek() << std::endl;  
// Output: 20
```

Dr. Mahmoud Alnamoly

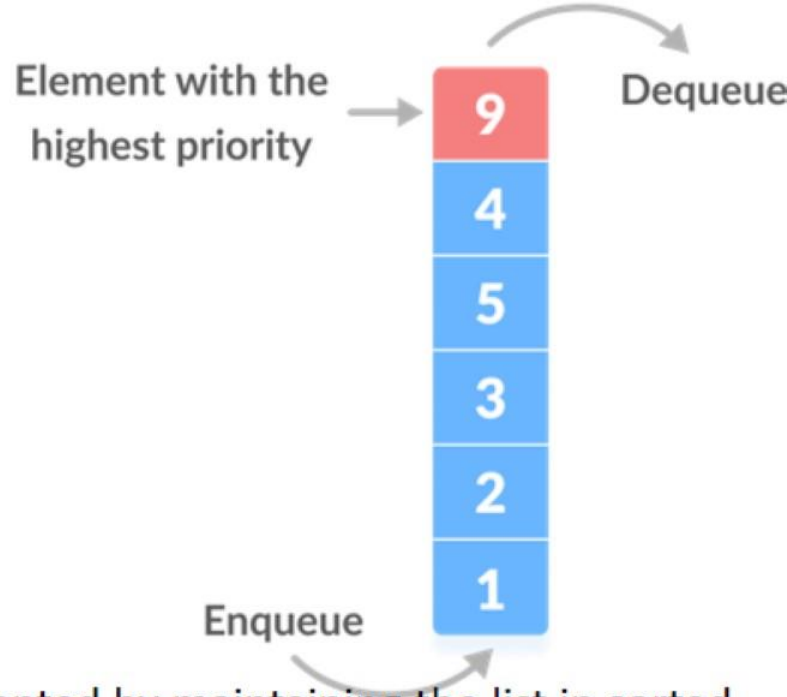
```
// Clear the queue
```

```
queue.clear();  
queue.display(); // Output: Queue is  
empty.
```

```
return 0;
```

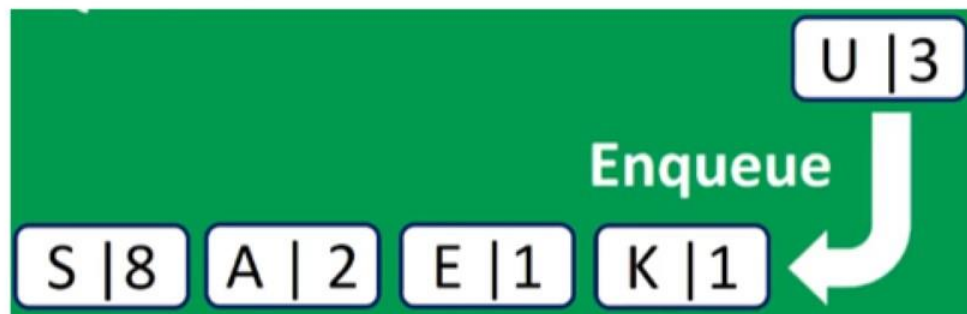
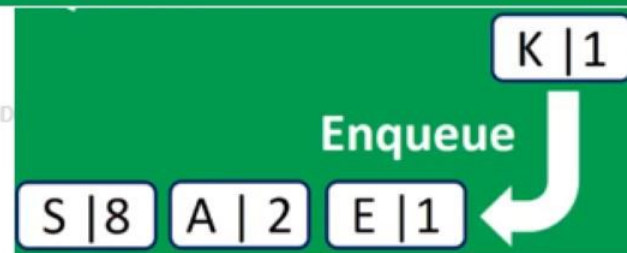
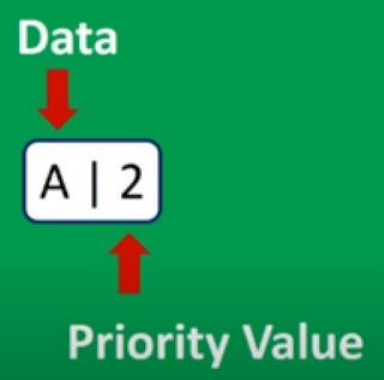

Priority Queue

- ✓ A priority queue is a type of data structure where each element has a **priority** assigned to it.
- ✓ Elements are processed based on their priority, with higher priority elements being processed before lower priority ones.
- ✓ This is different from a **regular queue** where elements are processed in the order they arrive (first-in, first-out).
- ✓ A Priority Queue using a **Linked List** can be implemented by maintaining the list in sorted order based on priority.

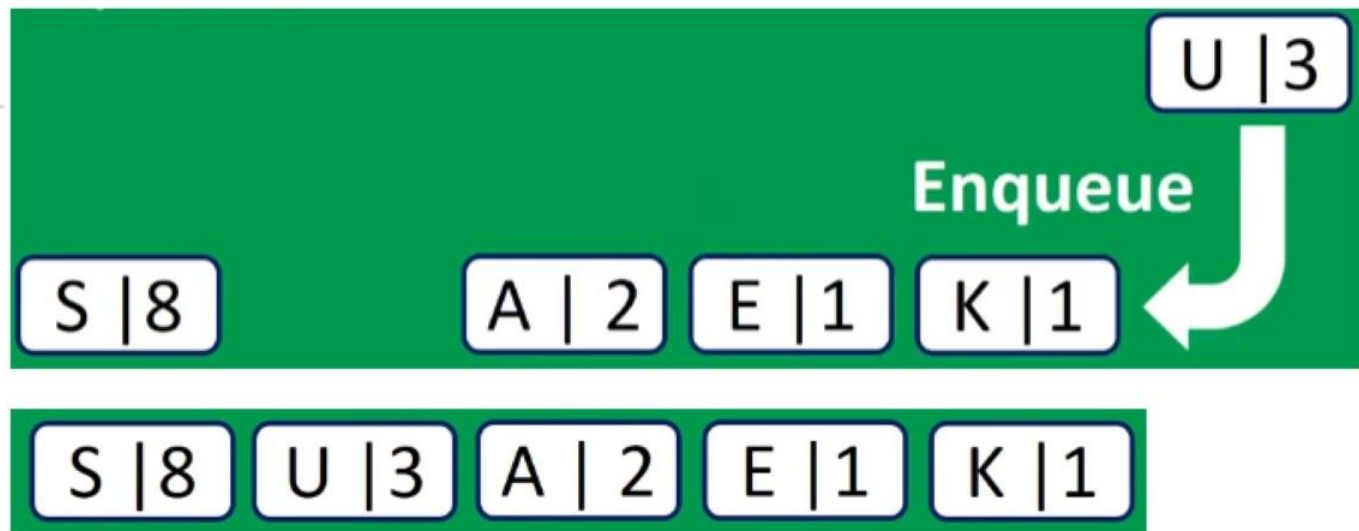


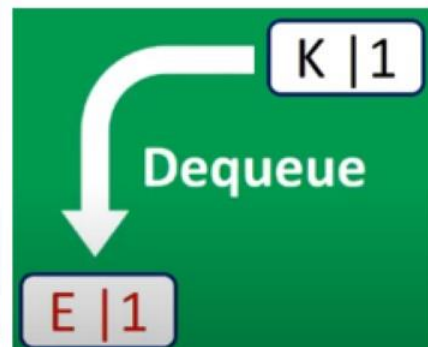
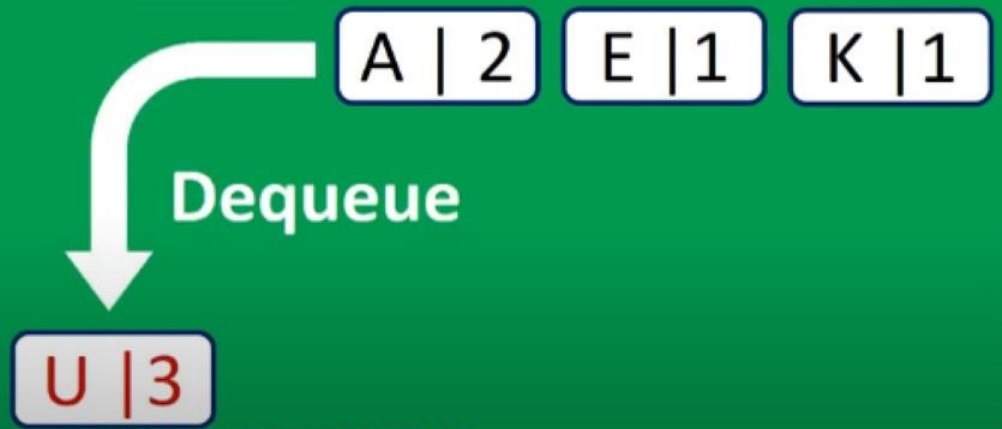
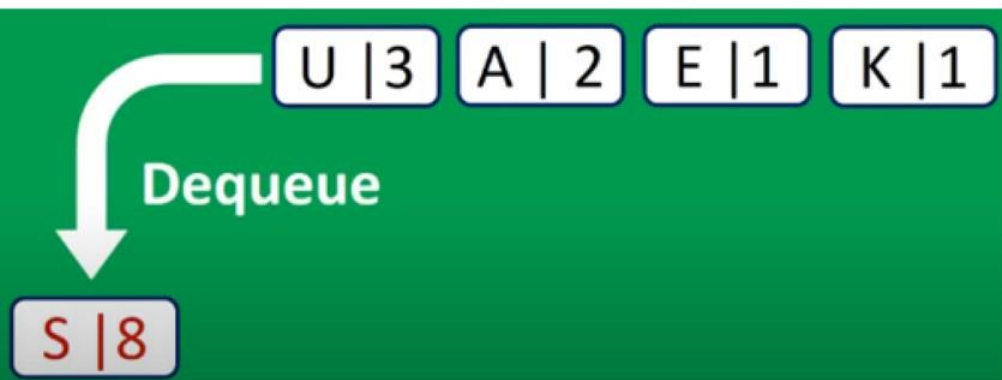
Insertion of a new element involves traversing the list to find the **appropriate position** and inserting the element there.

This ensures that the element with the **highest priority** (or smallest value in the case of a min-priority queue) is always **at the front of the list**.



Dr.





Priority Queue Using linked list

Dr. Mahmoud Alnamoly

```
#include <iostream>
using namespace std;

// Node structure for the linked list
template<class T>
struct Node {
    T data;
    int priority; //set priority for each element
    Node* next;

    Node(T value, int pri) {
        data = value;
        priority = pri;
        next = nullptr;
    }
};
```


Priority Queue Using linked list

```
// Priority Queue class using linked list
template<class T>
class PriorityQueue {
private:
    Node<T>* head; // Head node of the linked list

public:
    // Constructor to initialize an empty priority queue
    PriorityQueue() {
        head = nullptr;
    }

    // Destructor to free memory when the queue is no longer needed
    ~PriorityQueue() {
        while (head) {
            Node<T>* temp = head;
            head = head->next;
            delete temp;
        }
    }
};
```

Priority Queue Using linked list

Dr. Mahmoud Alnamoly

```
// Function to insert a value in the priority queue (sorted insertion)
```

```
void enqueue(T value,int pri) {
```

```
    Node<T>* newNode = new Node<T>(value,pri);
```

```
    // If the queue is empty or the new value has the highest priority
```

```
    if (!head || head->priority < pri) {
```

```
        newNode->next = head;
```

```
        head = newNode;
```

```
    }
```

Dr. Mahmoud Alnamoly

```
    else {
```

```
        // Traverse the list and find the correct position to insert
```

```
        Node<T>* temp = head;
```

```
        while (temp->next && temp->next->priority >= pri) {
```

```
            temp = temp->next;
```

```
        }
```

```
        newNode->next = temp->next;
```

```
        temp->next = newNode;
```

```
    }
```

```
    cout << value << " pushed into the priority queue.\n";
```

```
}
```

Dr. Mahmoud Alnamoly

Priority Queue Using linked list

Dr. Mahmoud Alnamoly

```
// Function to remove the element with the highest priority (smallest value)
void dequeue() {
    if (!head) {
        cout << "Priority queue is empty. Cannot pop.\n";
        return;
    }
    Node<T>* temp = head;
    cout << head->data << " popped from the priority queue.\n";
    head = head->next; // Move the head to the next node
    delete temp; // Free the memory of the popped node
}
```

Priority Queue Using linked list

Dr. Mahmoud Alnamoly

// Function to get the element with the highest priority (smallest value)

```
T top() {  
    if (!head) {  
        cout << "Priority queue is empty.\n";  
    }  
    return head->data;  
}
```

Dr. Mahmoud Alnamoly

// Function to check if the priority queue is empty

```
bool empty() {  
    return head == nullptr;  
}
```

Priority Queue Using linked list

Dr. Mahmoud Alnamoly

```
// Function to display all elements in the priority queue
void display() {
    if (!head) {
        cout << "Priority queue is empty.\n";
        return;
    }
    cout << "Priority queue elements: ";
    Node<T>* temp = head;
    while (temp) {
        cout << temp->data << " ";
        temp = temp->next;
    }
    cout << endl;
}
```


Priority Queue Using linked list

Dr. Mahmoud Alnamoly

```
int main() {
    PriorityQueue<std::string> pq;
    std::string name;
    int priority;
    cout << "Priority Queue rank from (1,20) \n";
    cout << "Enter patient name with his priority associated: \n";
    for (size_t i = 1; i < 5; i++)
    {
        cout << "Patient# "<<i<<"(Name Priority) : ";
        cin >> name >>priority;
        pq.enqueue(name, priority);
    }
    pq.display();
    cout << "Top element is: " << pq.top() << endl;
    pq.dequeue();
    pq.display();
    return 0;
}
```

Dr. Mahmoud Alnamoly

Output

Dr. Mahmoud Alnamoly

```
Priority Queue rank from (1,20)
Enter patient name with his priority associated:
Patient# 1(Name Priority) : aly 5
aly pushed into the priority queue.
Patient# 2(Name Priority) : aliaa 9
aliaa pushed into the priority queue.
Patient# 3(Name Priority) : mohamed 3
mohamed pushed into the priority queue.
Patient# 4(Name Priority) : aya 12
aya pushed into the priority queue.
Priority queue elements: aya aliaa aly mohamed
Top element is: aya
aya popped from the priority queue.
Priority queue elements: aliaa aly mohamed
```

Tree

>>> Next Lecture